

1 Executive Overview

SmartDrive AI is a browser-based intelligent driver safety simulator built as a single-page Next.js application. It models the complete safety loop of a modern assisted-driving system: the driver is monitored by an on-camera AI vision pipeline, a simulated sensor bus reports alcohol and helmet status, a safety engine evaluates risk ten times per second, and a vehicle physics layer moves the car, bike or truck along an animated road. When the safety engine decides that the driver is no longer fit to drive, it progressively takes over: it warns, slows the vehicle in a controlled stop, or performs a full autonomous roadside maneuver that steers the truck into a designated safe-stop zone with hazard lights flashing.

The project is designed as a tech-fest demonstration and an educational tool. Every detection system can be triggered manually from the control panel, so a presenter can stage a complete intervention story in under a minute without any real hardware. Six scripted demo scenarios automate exactly such sequences, from a bike that refuses to start without a helmet to a drowsy truck driver being parked safely at the roadside. An autopilot with cruise control and lane keeping, plus a high-speed drive profile, keeps the simulation moving on the road at all times with a single click.

The simulator is intentionally honest about its boundaries: all detection sources are clearly labeled, the helmet detector ships with a simulation fallback when no camera is available, and the on-screen warning states that the system is an educational simulation that never controls a real vehicle. No video ever leaves the browser - the MediaPipe face landmark model runs completely offline against a local WASM runtime.

Safety first. SmartDrive AI is an educational simulation. It demonstrates driver-monitoring and intervention concepts; it is not connected to any vehicle and must never be interpreted as functional driver-assistance software.

3

VEHICLE TYPES

6

DEMO SCENARIOS

60 fps

SIMULATION LOOP

10 Hz

SAFETY PULSE

2 Feature Walkthrough

2.1 Pre-Start Safety Interlocks

Starting a vehicle runs a 1.6-second pre-start checklist that mirrors real-world interlocks. The alcohol sensor reading is compared against a configurable threshold (default 40), and bikes additionally require a detected helmet. If any condition fails, START is blocked with a red banner naming the exact reason, the decision panel records a CRITICAL recommendation of BLOCK_START, and the condition must be cleared and the scenario reset before the engine will run. This turns the abstract idea of an interlock into something a viewer can watch happen step by step.

2.2 Driver Monitoring and Drowsiness

When camera access is granted, a MediaPipe FaceLandmarker model (bundled locally, 478 landmarks) analyzes the video stream at roughly 12 frames per second. Blendshape scores feed an eye-closure detector and a head-yaw proxy; a temporal logic layer converts raw closures into a drowsiness level with elapsed-closure time and a blink counter. Denying the camera never breaks the app: the camera panel shows a fallback state and every detection remains available through manual simulation buttons, which is also how the demo scenarios trigger events deterministically.

2.3 Risk Evaluation and Intervention

A pure-function safety engine receives the merged driver state every 100 milliseconds and classifies it as SAFE, WARNING or CRITICAL with an explainable decision record: severity, human-readable reason, recommended action, control mode and evidence list. Escalation follows vehicle-specific rules. A warned driver can keep driving; a critical finding triggers a controlled stop for cars and bikes, while the flagship truck performs a phased autonomous takeover - TAKEOVER, DECELERATION, ROADSIDE_ALIGNMENT, SAFE_STOP and STOPPED - planned by a trajectory planner that reserves a safe-stop zone on the shoulder and draws the intended path on the canvas. Accidents can be simulated at any speed and shake the viewport with a red flash while the session report records the impact speed.

2.4 Drive Modes: Manual, Autopilot, High Speed

After the engine starts, the vehicle can be driven with the keyboard (W/A/S/D or arrow keys, SPACE for an emergency stop) or handed to the AI chauffeur. AUTOPILOT applies a proportional cruise controller toward the vehicle's configured cruise speed and steers back to the lane center with a lateral lane-keeping loop. HIGH SPEED raises the cruise target to the vehicle's top speed and auto-engages autopilot if it is off, so one click always gets the vehicle moving on the road. Brake input, the P key, a stop request or any safety intervention disengages autopilot immediately - the safety engine always outranks the chauffeur.

2.5 Visualization, Telemetry and Reports

A top-down road canvas renders the live scene at 60 frames per second: streaming lane dashes, roadside trees and street lamps, speed streaks at highway pace, brake lights, headlight cones, hazard blinking and

the autonomous path overlay. Telemetry panels show speed, acceleration, roadside offset, steering, odometer and hazard state; a safety panel tracks drowsiness, helmet and alcohol status; and a decision panel explains why the system intervened. Every event lands in a deduplicated timeline, and at the end of a session a report summarizes counts, intervention details and results with JSON export for later analysis. An analytics view charts historical sessions and a settings view persists all thresholds to local storage.

3 Vehicle Lineup and Safety Matrix

Three vehicle archetypes cover the interesting safety combinations. The bike is the interlock story: it will not start without a helmet and loses part of its motor power when alcohol is detected. The car demonstrates the controlled-stop intervention path with the richest driver monitoring. The truck is the flagship: its size justifies the full autonomous roadside maneuver, including the trajectory planner and the reserved safe-stop zone on the shoulder.

Vehicle	Top speed	Cruise	Monitoring	Intervention behavior
BIKE	80 km/h	55 km/h	Helmet interlock, alcohol interlock, driver monitoring	Blocked start without helmet; power-reduction controlled stop when drunk
CAR	120 km/h	80 km/h	Driver monitoring, alcohol detection	Controlled stop on critical findings; accident response
TRUCK	90 km/h	68 km/h	Driver monitoring, alcohol detection	Autonomous takeover: decelerate, align to roadside, safe stop in reserved zone

Table 1: Vehicle specifications and safety behavior

Physics is deterministic and tuned for believability rather than realism: each vehicle has its own acceleration, brake deceleration, coasting drag and steering rate, with speed clamped to the specification. Lane geometry uses a 3.6-meter lane and a 4.6-meter shoulder; lateral position is constrained to the roadway plus a shoulder band, and heading is derived from lateral velocity so the sprite visibly tilts while changing lanes. The stop-distance helper that drives the autonomous planner is the same braking model the player experiences, which keeps the intervention distances honest.

4 System Architecture

The application is a deliberate three-layer loop: sense, decide and drive, then show. A single mutable engine object owns all simulation state and is ticked by one requestAnimationFrame loop, so there is exactly one source of truth. React never renders at frame rate; instead a Zustand mirror subscribes to a 10 Hz snapshot, while the road canvas reads the engine directly every animation frame for smooth motion. This split keeps the UI responsive and the animation fluid without threading state through React on every tick.

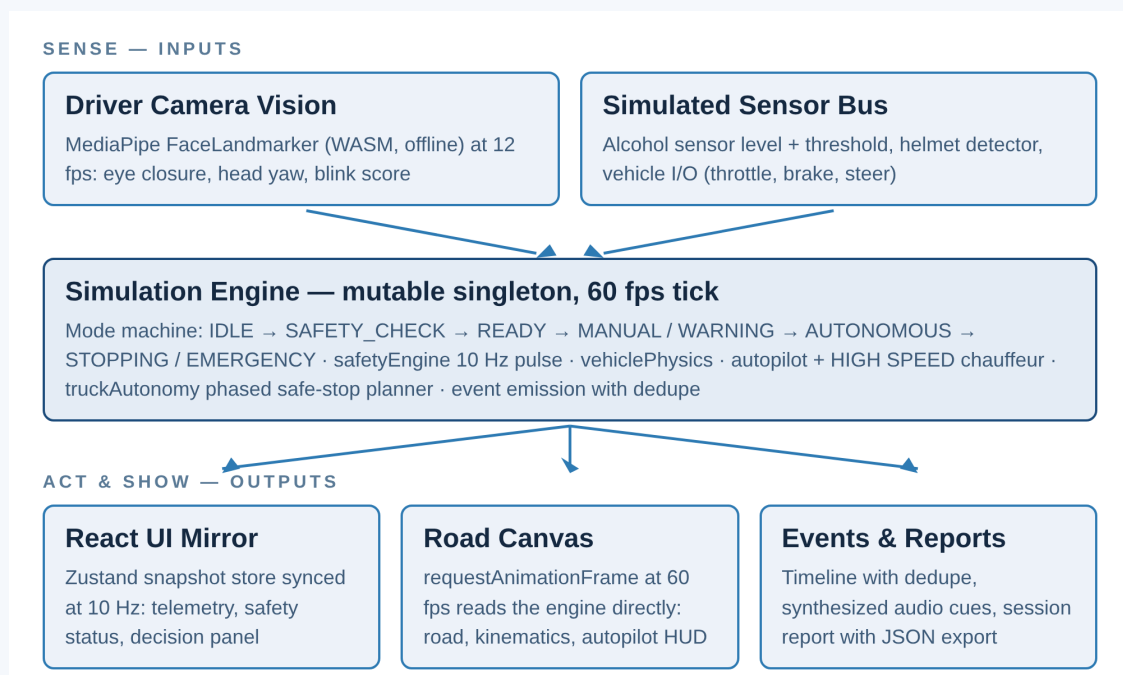


Figure 1: Sense - Decide & Drive - Show data flow with update frequencies

Module	Responsibility
engine.ts	Central state machine, 60 fps tick, mode transitions, autopilot and HIGH SPEED chauffeur, event emission with dedupe, session finalization
safetyEngine.ts	Pure evaluateSafety(): merges driver, alcohol, helmet and vehicle state into severity + explainable decision
vehiclePhysics.ts	Per-vehicle specs, stepPhysics() longitudinal + lateral integration, lane geometry, stopping-distance helper
truckAutonomy.ts	Phased takeover planner: takeover, deceleration, roadside alignment, safe stop in reserved zone
vehicleRules.ts	Declarative per-vehicle interlocks and intervention policy
scenarioEngine.ts	Six scripted demo scenarios with condition-based step advance
faceVision.ts / helmet.ts / drowsiness.ts	MediaPipe landmarker wrapper with GPU-to-CPU fallback, pluggable helmet detector with simulation fallback, temporal drowsiness logic

Module	Responsibility
store.ts	Zustand mirror: 10 Hz snapshot sync, engine command wrappers, demo runner, settings persistence

Table 2: Core modules and responsibilities

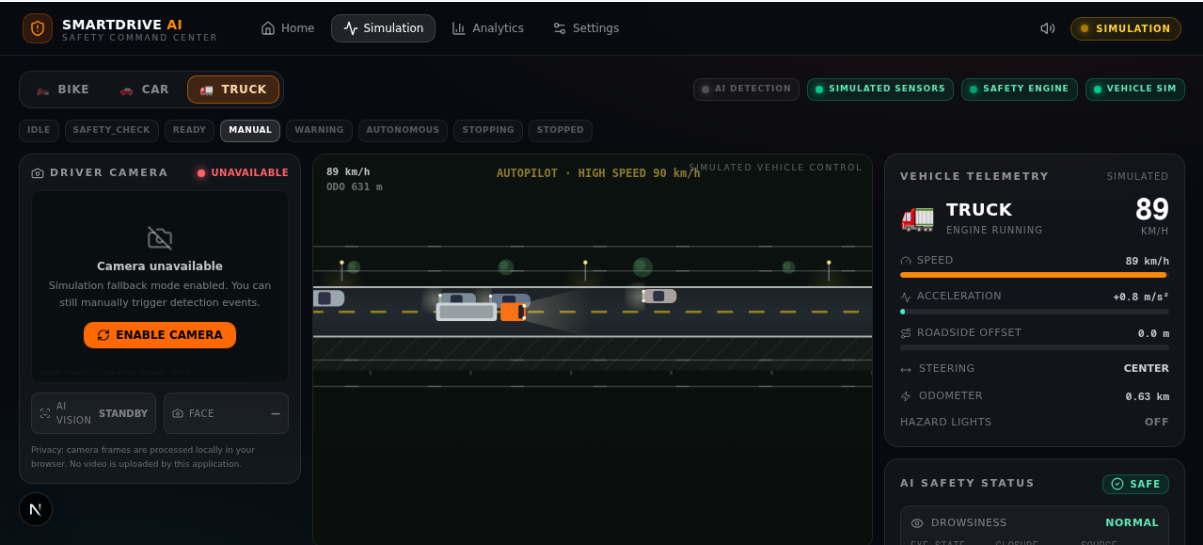
5 Drive Modes and Controls

The Simulation Controls panel groups the two automation buttons directly under the primary START / STOP / RESET / EMERGENCY row. AUTOPILOT toggles the cruise chauffeur; HIGH SPEED raises the target to top speed and engages autopilot in one action. Both buttons disable themselves outside manual-driving states, during safety interventions and while a demo scenario owns the vehicle, and a status line under the buttons always explains the current drive mode and how to leave it. A pulsing hint invites the user to press AUTOPILOT or HIGH SPEED whenever the vehicle is at standstill.

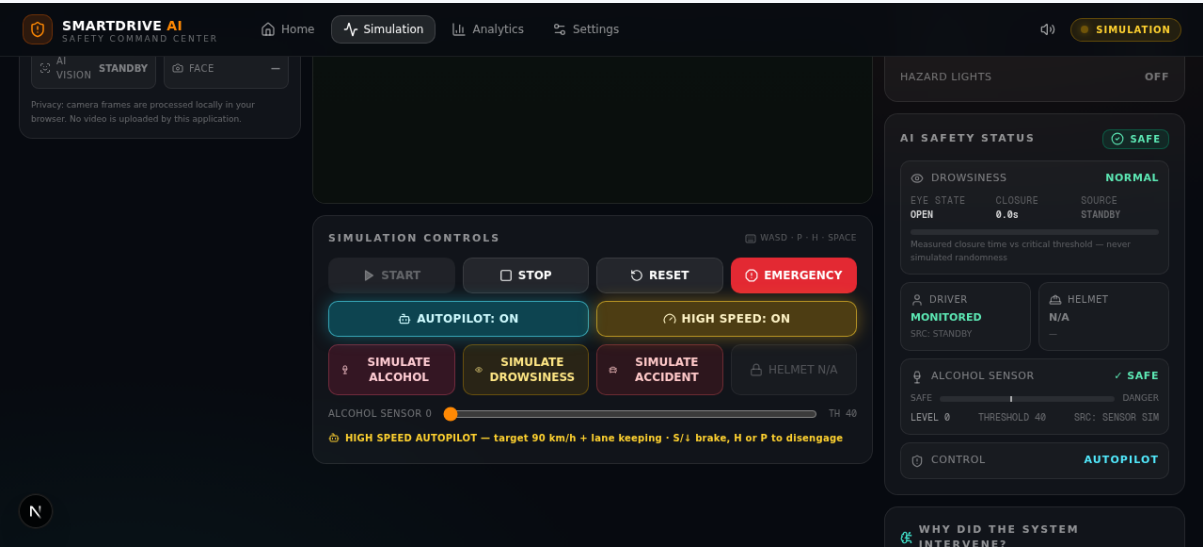
Control	Action
W / Arrow Up	Throttle (hold)
S / Arrow Down	Brake (hold) - also disengages autopilot above 20% input
A / D or Arrow Left / Right	Steer left / right
SPACE	Emergency stop
P	Autopilot on / off
H	High-speed profile on / off
AUTOPILOT button	Cruise at configured speed + lane keeping
HIGH SPEED button	Target top speed (CAR 120 / TRUCK 90 / BIKE 80 km/h), auto-engages autopilot

Table 3: Keyboard and drive-mode reference

Disengagement follows real ADAS conventions. Any safety intervention, emergency stop, operator stop request or deliberate brake input instantly returns control to the driver and clears the high-speed flag, and the timeline records who gave up control and why. During autonomous takeover the engine locks manual input altogether until RESET starts a fresh scenario. Screenshot 1 shows the road scene under autopilot with the lane-keeping offset held at 0.0 meters; screenshot 2 shows the drive-mode buttons engaged at their targets.



Screenshot 1: Road scene under autopilot - live canvas with scenery and HUD



Screenshot 2: Drive-mode buttons engaged - AUTOPILOT ON and HIGH SPEED ON

6 Demo Scenarios

Six one-click scenarios provide a judge-ready storyline. Each scenario runs a scripted step list whose steps advance only when the real simulation state meets their advance condition (with timeout fallbacks), so the demo cannot drift out of sync. While a scenario owns the session, automation toggles are locked and a progress bar shows the active step.

#	Scenario	Vehicle	Trigger	Outcome
1	Bike Helmet Safety	BIKE	Helmet removed	START blocked until helmet is worn
2	Bike Alcohol Prevention	BIKE	Alcohol above threshold	Start blocked; with engine running, power-limited controlled stop

#	Scenario	Vehicle	Trigger	Outcome
3	Car Drowsiness Stop	CAR	Simulated eye closure	Warning escalates to controlled stop (about 6 s)
4	Truck Alcohol Autonomous Stop	TRUCK	Alcohol above threshold	Phased takeover, roadside alignment, safe stop with hazards
5	Truck Drowsiness Autonomous Stop	TRUCK	Drowsiness critical	Autonomous safe stop in reserved zone with path overlay
6	Truck Accident Emergency Stop	TRUCK	Simulated accident	Viewport shake, emergency response, session report records impact

Table 4: Scripted demo scenarios

7 Tech Stack and Project Structure

The stack is a modern, fully client-side Next.js application. Next.js 16 with React 19 and TypeScript provides the app shell; Tailwind CSS 4 plus a set of Radix-based shadcn/ui components style the dark automotive theme; framer-motion animates banners and transitions; Recharts powers the analytics view; Zustand mirrors engine state into React; and lucide-react supplies icons. The vision pipeline uses the official MediaPipe tasks-vision package with the face landmarker model and its WASM runtime vendored into public/, so the entire system runs offline after install. Prisma appears in the template scaffold but the simulator itself persists only to localStorage.

Layer	Technology
App framework	Next.js 16 (App Router), React 19, TypeScript
UI	Tailwind CSS 4, shadcn/ui (Radix), framer-motion, lucide-react
State	Zustand snapshot mirror (10 Hz) over a mutable engine singleton
Simulation	Custom engine: physics, safety engine, autonomy planner, scenario engine (pure TypeScript)
AI vision	MediaPipe tasks-vision FaceLandmarker, WASM runtime + model vendored locally, GPU-to-CPU fallback
Charts / analytics	Recharts
Persistence	localStorage (settings, session history); no server database required
Tooling	bun scripts, ESLint, tsc strict typecheck, Playwright E2E checks

Table 5: Technology map

```
src/  
  app/                                layout, single-route SPA shell, global theme  
  components/sim/                     RoadCanvas, ControlPanel, TelemetryPanel,  
                                      SafetyPanel, CameraPanel, DemoPanel,  
                                      EventTimeline, SessionReport, VehicleSelector,  
                                      AlertBanner / DecisionPanel / ModeStrip  
  hooks/                             useSimulationLoop (rAF), useVision (camera + model),  
                                      useKeyboard (WASD / P / H / SPACE)  
  lib/simulation/                    engine, safetyEngine, vehiclePhysics,  
                                      truckAutonomy, vehicleRules, scenarioEngine  
  lib/  
public/mediapipe/wasm/               vendored WASM vision runtime  
public/models/                       face_landmarker.task (local, offline)
```

Listing 1: Project layout (key paths)

8 Running and Verifying the Project

The project runs like any Next.js application: install dependencies with bun, start the development server on port 3000 and open the single-route app. The landing page introduces the concept; the Simulation view hosts the live dashboard; Analytics charts past sessions; Settings exposes every threshold with persistence. A typical demonstration takes under a minute: select the truck, press START, watch the safety checks pass, press HIGH SPEED and let the vehicle cruise, then press SIMULATE DROWSINESS to hand the story over to the safety engine.

```
bun install          # install dependencies  
bun run dev          # dev server at http://localhost:3000  
bun run lint         # eslint (clean)  
npx tsc --noEmit     # strict typecheck (clean)  
bun run build        # production build
```

Behavior was verified end-to-end in a real browser with automated UI walks. The vehicle-acceleration fix was confirmed by holding the throttle and watching speed rise, then decay only by the specified coasting drag; the autopilot and high-speed modes were verified by engaging each button and reading live telemetry; brake-key disengagement, re-engagement and RESET behavior were each exercised. The most recent verification pass is summarized below.

Check	Expected	Observed
Pre-start interlock	Bike blocks START without helmet	Blocked with reason banner
Autopilot cruise	Truck converges to 68 km/h, offset 0.0 m	68 km/h held, offset 0.0 m
High-speed profile	Target rises to truck top speed	90 km/h reached and held
Brake disengagement	AP exits on brake, event logged	Disengaged; timeline event recorded
Manual keyboard drive	W accelerates from standstill	0 to 24 km/h in 3 s (truck)
Static checks	ESLint + tsc clean	Both clean; no page errors on load

Table 6: Latest browser verification pass

Natural next steps for the simulator include optional ambient traffic behind a settings toggle, richer lane geometry with curves, an import/export format for session reports, and a comparison mode that replays the same scenario with and without intervention to quantify the safety benefit. The engine's pure-function safety core and single-source-of-truth design keep all of these additions incremental rather than architectural.